

DocsMCP

Build Document · v1.0

Turn any documentation site into an MCP endpoint — on demand.

What We Are Building

DocsMCP is a hosted service that takes any public documentation URL, crawls it, embeds it into a vector store, and exposes a personal MCP (Model Context Protocol) endpoint that Claude, Cursor, or any MCP-compatible client can query in real time.

The developer never pastes docs into the chat again. They crawl once, get an MCP endpoint, and their AI assistant answers from the actual docs — not hallucinated memory.

Flow

URL → Crawl → Chunk → Embed → Vector DB → MCP Endpoint → Claude / Cursor

Core Features (MVP)

- Submit any public docs URL — get an MCP endpoint back
 - On-demand crawl — not pre-crawled, not a catalog
 - Cache sharing — two users crawling the same URL reuse the same vectors
 - API key auth per endpoint — each crawl gets its own bearer token
 - Polling endpoint to track crawl status
 - `search_docs` MCP tool — semantic search over the crawled site
 - `get_page` MCP tool — fetch full content of a specific doc page
-

Architecture Overview

Single Go binary (MVP) serving both the REST API and the MCP server. Redis for queuing and caching. Qdrant for vectors. Postgres for metadata.

```
User → POST /crawl → Job Queue → Crawler → Chunker → Embedder
→ Qdrant

↓
Claude / Cursor → MCP Endpoint ← MCP Server
←
```

Components

1. REST API

- POST /crawl — accepts { url }, returns { job_id, api_key, mcp_endpoint, status }
- GET /crawl/:job_id — poll for status: queued → crawling → chunking → embedding → ready → failed
- GET /mcps — list all crawls for the authenticated user
- DELETE /mcps/:job_id — remove a crawl
- URL normalization — strip trailing slashes, query params, fragments before hashing

2. Job Queue

- Asynq (Go-native Redis-backed queue)
- User hits API → job enqueued → worker picks up async
- Job states tracked in both Postgres and Redis
- Retry on transient failures — max 3 retries with backoff

3. Crawler — Fallback Chain

This is the most critical component. Try each strategy in order, fall through on failure or empty content:

1

Plain HTTP + goquery

Fastest. Works for static sites (Hugo, Jekyll, plain HTML). Extract text via CSS selectors targeting main content areas.

2

go-rod (headless Chromium)

For JS-rendered sites — Docusaurus, VitePress, Next.js docs. Detects JS-heavy pages by low text-to-script ratio in response.

3**Jina Reader API**

<https://r.jina.ai/<url>> — free, returns clean markdown. Use when headless is too slow or bot-detected.

4**Firecrawl API**

Paid, last resort. Handles anti-bot, auth-walled, JS-heavy pages that everything else fails on.

Additional crawler rules:

- Check sitemap.xml first — massively speeds up page discovery
- Stay on the same domain — do not follow external links
- Prioritize paths containing /docs/, /api/, /guide/, /reference/
- Hard cap: 500 pages per crawl in MVP
- Respect robots.txt
- Skip: PDFs, images, changelogs, blog posts (configurable filter)
- Rate limit outgoing requests — max 5 concurrent per crawl job

4. Content Extractor

- Use go-readability to strip nav, sidebar, footer, ads from HTML
- Output: clean text + code blocks, preserving semantic structure
- Tag each page with: url, title, heading hierarchy, section type

5. Chunker

- Split by heading hierarchy first (h1 → h2 → h3)
- Code blocks stay intact — never split mid-code
- Target: 400–600 tokens per chunk with ~50 token overlap
- Each chunk carries metadata: source_url, page_title, heading_path, crawl_id, chunk_index
- Code-heavy pages (API reference) get larger chunk limit — up to 800 tokens

6. Embedder

- OpenAI text-embedding-3-small — cheap, fast, good retrieval quality
- Batch embed — 100 chunks per API call, never one at a time
- Store: vector + metadata in Qdrant, raw chunk text alongside

7. Vector Store — Qdrant

- One collection per normalized URL hash — shared across all users who crawl same site
- Collection name: docs_{md5(normalized_url)[:12]}

- Each point: vector (1536d) + payload { crawl_id, url, title, heading, text, chunk_index }
- Qdrant self-hosted on same VPS for MVP (no extra cost)

8. MCP Server

- Route: /mcp/:crawl_id — logical separation per crawl
- Auth: Authorization: Bearer <api_key> checked before every tool call
- MCP tool: search_docs(query, top_k=5) — vector search, returns chunks + source URLs
- MCP tool: get_page(url) — returns full content of a specific page from the crawl
- Returns MCP-compliant JSON (follows MCP spec tool result format)

9. Cache Layer — Redis

- url_hash → crawl_id mapping (cache hit = skip crawl, return existing endpoint)
 - Job status key per job_id — fast polling without hitting Postgres
 - Rate limiting per user API key
-

Data Layer

Postgres Schema

```
-- Users (platform auth)
users(id, email, platform_api_key, created_at)

-- One row per unique crawled URL
crawls(id, url_raw, url_normalized, url_hash,
       status, page_count, chunk_count,
       qdrant_collection, created_at, ready_at)

-- Links users to crawls, holds their MCP key
user_crawls(id, user_id, crawl_id, mcp_api_key, created_at)

-- Individual pages within a crawl
crawl_pages(id, crawl_id, url, title, chunk_count, crawled_at)
```

Key Design

crawls table is keyed on url_hash — shared between users. user_crawls holds each user's unique mcp_api_key for the same underlying crawl.

Qdrant Collection Structure

- Collection per crawl: docs_<url_hash[:12]>
 - Vector size: 1536 (text-embedding-3-small)
 - Distance: Cosine
 - Payload fields: crawl_id, page_url, page_title, heading_path, text, chunk_index
 - On cache hit: new user_crawls row pointing to existing collection — no re-embedding
-

Build Plan — Phase by Phase

Phase 0 — Project Setup

Get the skeleton running before any real logic.

1. Create Go module: `go mod init docsmcp`
2. Directory structure — see below
3. Docker Compose: Postgres + Redis + Qdrant (local dev)
4. Environment config — `.env` with all keys (OpenAI, Firecrawl, DB URLs)
5. Basic HTTP server boots, `/health` returns 200
6. Database migrations tool — use `golang-migrate`
7. Run migrations, confirm tables created

```
docsmcp/  
cmd/server/main.go      # entrypoint  
internal/  
  api/                  # HTTP handlers  
  crawler/              # fallback chain  
  chunker/              # text splitting  
  embedder/             # OpenAI calls  
  store/                # Qdrant + Postgres  
  mcp/                  # MCP server  
  worker/               # Asynq job handlers  
migrations/             # SQL files  
docker-compose.yml
```

Phase 1 — Crawler

Build and test the fallback crawler in isolation before wiring it to the queue.

8. Implement URL normalizer — lowercase, strip trailing slash, canonical form
9. Implement plain HTTP fetch + goquery extractor (Strategy 1)
10. Add JS detection — if text/script ratio below threshold, flag as JS-heavy
11. Implement go-rod headless fetcher (Strategy 2) — extract rendered DOM
12. Implement Jina Reader fallback (Strategy 3) — simple HTTP to `r.jina.ai`
13. Implement Firecrawl fallback (Strategy 4) — API call
14. Build the fallback orchestrator — try each in order, stop on success
15. Implement sitemap.xml parser — seed URL queue from sitemap if available
16. Build link extractor — find all same-domain doc links from a page
17. Implement crawl queue (in-memory BFS for now) with visited tracking
18. Add domain filter — skip external links, skip non-doc paths
19. Add page cap — stop at 500 pages
20. Write CLI test tool: crawl a URL, print page count + sample content

**Test
Target**

Use <https://docs.pocketbase.io> — clean static site, good for Strategy 1. Then test <https://react.dev> — JS-rendered, needs Strategy 2.

Phase 2 — Chunker

21. Implement HTML → clean text extractor using go-readability
22. Build heading-aware splitter — respects h1/h2/h3 boundaries
23. Implement code block protection — never split inside a code block
24. Add token counter — use tiktoken-go to measure chunk size
25. Implement overlap logic — 50 token overlap between adjacent chunks
26. Attach metadata to each chunk: url, title, heading_path, chunk_index
27. Unit test: verify no chunk exceeds 800 tokens, code blocks are intact

Phase 3 — Embedder + Vector Store

28. Set up OpenAI client — text-embedding-3-small, batch size 100
29. Set up Qdrant client — create collection helper with correct config
30. Implement embed + upsert pipeline: chunks → embeddings → Qdrant points
31. Implement search function: query string → embedding → Qdrant search → top-k chunks
32. Test: crawl PocketBase docs, embed, run 5 search queries, verify relevance

Phase 4 — Job Queue + Worker

33. Add Asynq to the project — client for enqueueing, server for processing
34. Define CrawlTask payload: { crawl_id, url, user_id }
35. Implement worker handler: pull task → run crawler → chunker → embedder → update DB
36. Add job status updates throughout — write to both Postgres and Redis
37. Implement error handling — mark job as failed with error message on unrecoverable error
38. Add retry logic — Asynq handles this, configure max 3 retries

Phase 5 — REST API

39. POST /crawl — validate URL, check Redis cache, create crawl + user_crawls row, enqueue job, return response
40. GET /crawl/:job_id — read status from Redis (fast), fall back to Postgres
41. GET /mcps — list user's crawls with status and endpoint URL
42. DELETE /mcps/:job_id — soft delete user_crawls row
43. Platform auth middleware — validate platform API key from header

- 44. Rate limiting middleware — max 10 crawls per user per hour
- 45. Test all endpoints with curl

Phase 6 — MCP Server

- 46. Study MCP spec — understand tool definition, tool call, tool result JSON format
- 47. Implement MCP route handler at /mcp/:crawl_id
- 48. Add auth middleware — validate mcp_api_key from Authorization header
- 49. Implement search_docs tool — receive query, run Qdrant search, format as MCP result
- 50. Implement get_page tool — receive URL, return full page text from crawl
- 51. Return proper MCP tool schema in capabilities response
- 52. Test by connecting to Claude Desktop — add to claude_desktop_config.json and run a query

MCP Config

```
{ "mcpServers": { "pocketbase": { "url": "https://your-server.com/mcp/<crawl_id>",  
  "headers": { "Authorization": "Bearer <api_key>" } } }
```

Phase 7 — Deploy

- 53. Provision a VPS — Railway or Fly.io, 2GB RAM is enough for MVP
 - 54. Write Dockerfile — single binary, copy static assets
 - 55. Docker Compose for production: app + Qdrant + Redis + Postgres
 - 56. Set all environment variables in the platform
 - 57. Set up domain + TLS (Fly handles this automatically)
 - 58. Deploy, run migrations, smoke test all endpoints
 - 59. Connect one real docs site end-to-end from production
-

Full Tech Stack

Component	Choice	Reason
API + MCP Server	Go (net/http)	Single binary, low memory, fast startup
Job Queue	Asynq + Redis	Go-native, reliable, great retry support
Crawler — S1	go-rod / chromedp	Headless Chrome for JS-heavy docs
Crawler — S2	Jina Reader API	Free fallback, returns clean markdown
Crawler — S3	Firecrawl API	Paid last resort, handles anything
Content Extract	go-readability	Strips nav/footer, returns main content
Chunker	Custom + tiktoken-go	Heading-aware, code-block-safe splitting
Embeddings	OpenAI embedding-3-small	Cheap (\$0.02/1M tokens), high quality
Vector DB	Qdrant (self-hosted)	Fast, great Go client, free to self-host
Metadata DB	Postgres	Source of truth for crawls + users
Cache	Redis	Job status, URL→crawl cache, rate limits
Deploy	Fly.io	Simple, auto-TLS, good free tier for MVP

What to Skip in MVP

Skip	Re-crawl scheduling / staleness detection — user manually re-triggers if needed
Skip	Multi-version docs — crawl one version only (whatever the URL points to)
Skip	Auth-walled documentation — only public sites in v1
Skip	Webhook on crawl complete — polling via GET /crawl/:job_id is sufficient
Skip	Frontend UI — API-only in MVP, developers use curl / Postman
Skip	Your own embedding model — OpenAI API is fast and cheap enough at this scale

Known Hard Parts — Watch Out

Crawl Scope Explosion

Docs sites link to GitHub, npm, external tutorials. Without strict same-domain + path filtering, a single crawl can try to index the entire internet. Enforce domain lock and path prefix from the start.

go-rod Memory

Headless Chrome is heavy. Each go-rod instance uses ~150MB. Run max 2 concurrent headless crawls per machine in MVP. Kill the browser instance immediately after each page is fetched.

Qdrant Collection Naming

URL hashes must be deterministic and stable. Normalize the URL before hashing: lowercase domain, remove trailing slash, remove www prefix, sort query params (then drop them for doc sites).

Chunk Quality

Heading-based splitting assumes well-structured HTML. Some docs (especially auto-generated API references) are flat. Add a fallback size-based splitter for pages with no h2/h3 headings.

MCP Auth

The `mcp_api_key` is sensitive — it grants read access to an embedded doc site. Store it hashed in Postgres (bcrypt), send plaintext only once at creation time. Do not log it.

Day 1 — Start Here

Finish these before writing any crawler or embedding code:

60. `go mod init docsmcp`
61. Create directory structure as shown in Phase 0
62. Write `docker-compose.yml` with Postgres (5432), Redis (6379), Qdrant (6333)
63. `docker compose up` — confirm all three services are running

64. Write first migration: create users, crawls, user_crawls, crawl_pages tables
65. Run migration — confirm in psql
66. Write main.go — HTTP server on :8080, /health endpoint returning { ok: true }
67. curl localhost:8080/health — confirm it works
68. Commit. You now have a working skeleton.
69. Start Phase 1 — build plain HTTP crawler, test on docs.pocketbase.io

Rule

Do not write the MCP server until the crawler + embedder pipeline produces real, searchable results. Validate the data quality first.

DocsMCP Build Document · Start building.